

LAPORAN PENAMBAHAN INHERITANCE PADA CLASS DIAGRAM

PEMROGRAMAN BERORIENTASI OBJEK

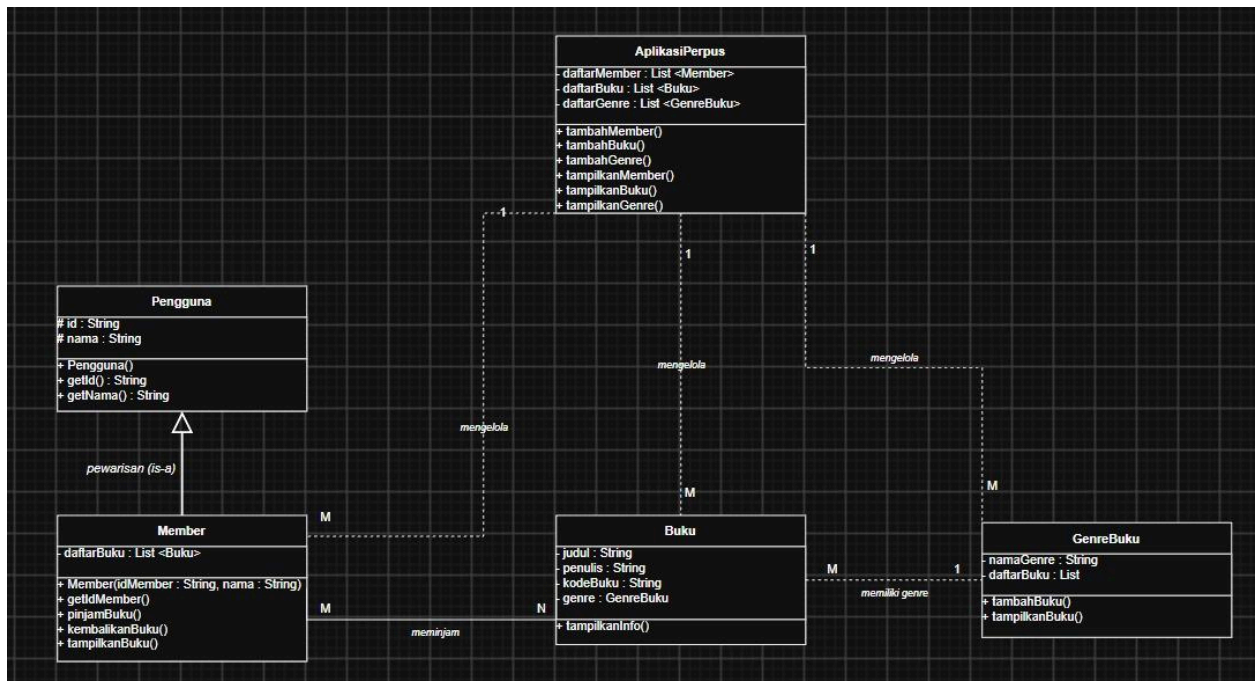
Kelompok 1

Nama Anggota:

Muhammad Aditya Firmansyah	J0403251035
Ibrahim Ahmad Idhofi	J0403251107
Rifqi Tazakka Putra	J0403251158
Alyaa Mardlatillah Sofwan	J0403251148
Irdina Nafeesa Ariyadi	J0403251041

1. Class Diagram

Berikut merupakan class diagram aplikasi perpustakaan setelah dilakukan penambahan konsep inheritance.



Class diagram pada aplikasi perpustakaan menggambarkan struktur class yang digunakan dalam program beserta hubungan antarclass. Class yang terdapat dalam aplikasi ini meliputi **Pengguna**, **Member**, **Buku**, **GenreBuku**, dan **AplikasiPerpus**.

Pada pengembangan aplikasi, ditambahkan konsep inheritance antara class Pengguna dan Member. Class Pengguna berperan sebagai superclass, sedangkan Member menjadi subclass yang mewarisi atribut dan method dari Pengguna.

Hubungan tersebut diterapkan pada code Java menggunakan keyword extends sehingga Member memiliki hubungan pewarisan dengan Pengguna.

2. Penjelasan Class Diagram

2.1 Class Pengguna

Class Pengguna merupakan superclass yang digunakan untuk menyimpan data umum yang dimiliki oleh pengguna. Class ini memiliki atribut id dan nama.

Atribut tersebut menggunakan access modifier protected sehingga dapat diakses oleh class turunan seperti Member.

Class Pengguna juga memiliki beberapa method, yaitu getId() untuk mengambil ID dan getName() untuk mengambil nama pengguna.

2.2 Class Member

Class Member merupakan subclass dari Pengguna. Hubungan inheritance diterapkan melalui kode:

```
public class Member extends Pengguna
```

Dengan adanya inheritance, Member dapat menggunakan atribut dan method yang berasal dari Pengguna.

Selain atribut yang diwariskan, Member memiliki atribut daftarBuku yang digunakan untuk menyimpan daftar buku yang sedang dipinjam oleh member.

Beberapa method yang terdapat pada class Member antara lain:

- getIdMember()
- getName()
- pinjamBuku()
- kembalikanBuku()
- tampilkanBuku()

2.3 Class Buku

Class Buku digunakan untuk menyimpan informasi mengenai buku yang terdapat dalam aplikasi perpustakaan.

Atribut yang dimiliki antara lain judul buku, nama penulis, kode buku, dan genre. Class ini juga memiliki method `tampilkanInfo()` yang digunakan untuk menampilkan informasi mengenai buku.

2.4 Class GenreBuku

Class `GenreBuku` digunakan untuk mengelompokkan buku berdasarkan genre tertentu.

Class ini memiliki atribut `namaGenre` dan `daftarBuku`. `namaGenre` digunakan untuk menyimpan nama kategori buku, sedangkan `daftarBuku` digunakan untuk menyimpan kumpulan buku yang termasuk dalam kategori tersebut.

Class ini juga memiliki method untuk menambahkan dan menampilkan daftar buku berdasarkan genre.

2.5 Class AplikasiPerpus

Class `AplikasiPerpus` berfungsi untuk mengelola data utama dalam aplikasi perpustakaan.

Class ini menyimpan daftar member, buku, dan genre melalui beberapa atribut berupa List.

Beberapa method yang terdapat pada class ini adalah:

- `tambahMember()`
- `tambahBuku()`
- `tambahGenre()`
- `tampilkanMember()`
- `tampilkanBuku()`
- `tampilkanGenre()`

Class `AplikasiPerpus` menjadi bagian yang mengatur data-data yang digunakan dalam sistem perpustakaan.

3. Penambahan Inheritance

Penambahan inheritance dilakukan dengan membuat class baru bernama `Pengguna` sebagai superclass. Selanjutnya, class `Member` dijadikan sebagai subclass dari `Pengguna`.

Sebelum dilakukan perubahan, data seperti ID dan nama berada langsung di dalam class `Member`. Setelah inheritance diterapkan, data yang bersifat umum dipindahkan ke class `Pengguna`.

Hubungan inheritance tersebut diterapkan dengan menggunakan keyword extends:
`public class Member extends Pengguna`

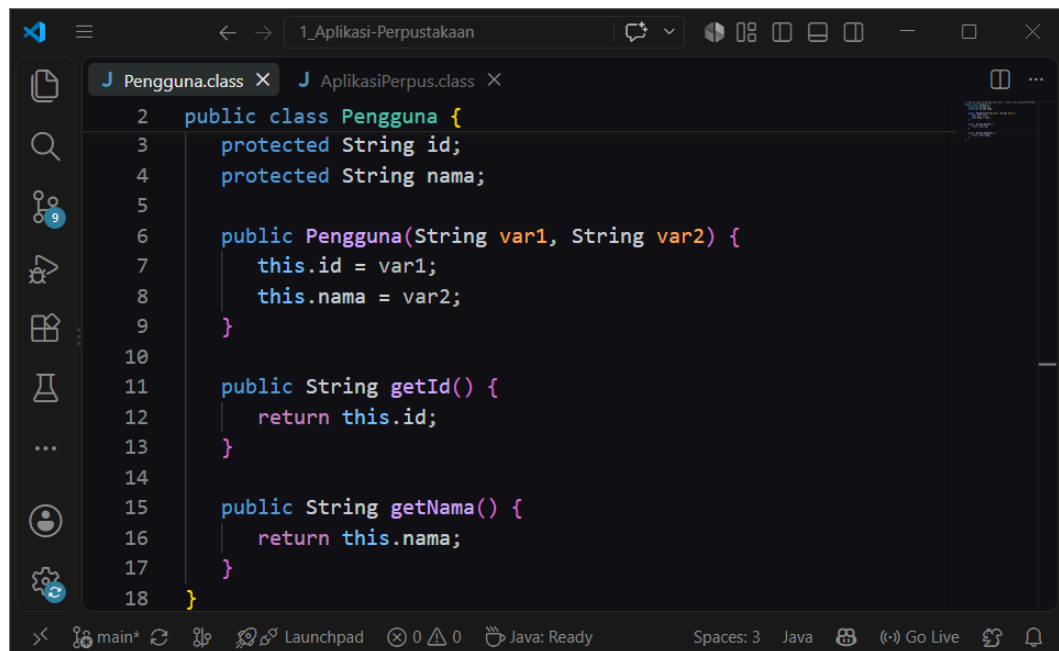
Dengan perubahan ini, Member dapat mewarisi atribut dan method yang terdapat pada Pengguna. Sementara itu, fungsi yang secara khusus berkaitan dengan member, seperti peminjaman dan pengembalian buku, tetap berada di dalam class Member.

Penerapan inheritance ini membuat struktur program menjadi lebih terorganisir karena data umum dan fungsi khusus dapat dipisahkan berdasarkan tanggung jawab masing-masing class.

4. Penjelasan Code

4.1 Class Pengguna

Class Pengguna digunakan sebagai superclass dan berisi data umum berupa ID dan nama.



```
2 public class Pengguna {
3     protected String id;
4     protected String nama;
5
6     public Pengguna(String var1, String var2) {
7         this.id = var1;
8         this.nama = var2;
9     }
10
11     public String getId() {
12         return this.id;
13     }
14
15     public String getNama() {
16         return this.nama;
17     }
18 }
```

Atribut id dan nama menggunakan modifier protected. Penggunaan modifier tersebut memungkinkan atribut diakses oleh class turunan.

4.2 Penggunaan Keyword extends

Inheritance pada class Member ditunjukkan dengan penggunaan keyword extends.

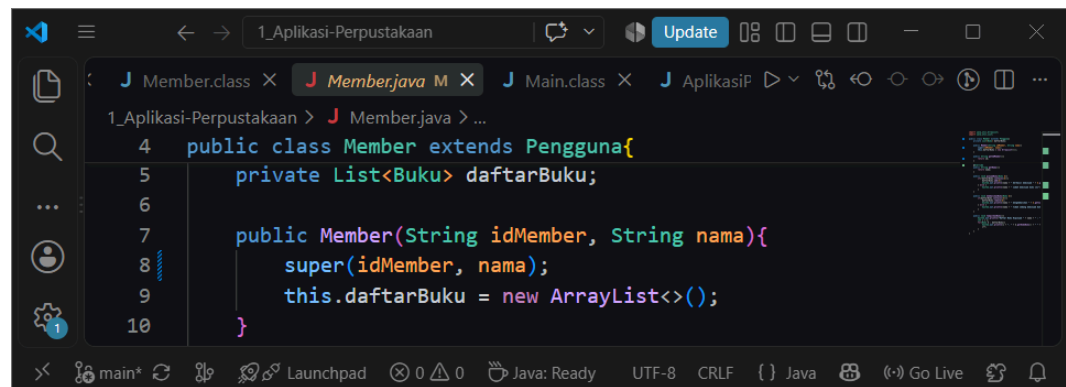
`public class Member extends Pengguna`

Kode tersebut menunjukkan bahwa class Member merupakan turunan dari class Pengguna.

Dengan demikian, atribut dan method yang dimiliki oleh Pengguna dapat digunakan oleh Member tanpa perlu dibuat ulang.

4.3 Penggunaan super()

Pada constructor Member terdapat penggunaan super() sebagai berikut:



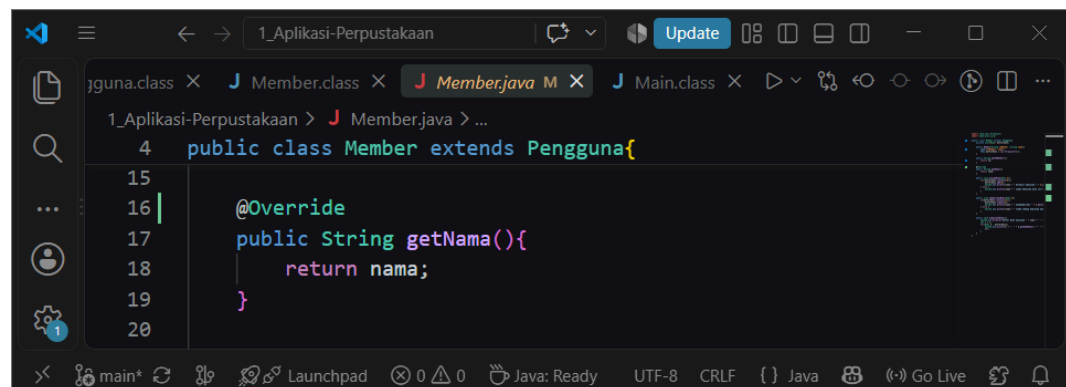
```
1_Aplikasi-Perpustakaan
J Member.class x J Member.java M x J Main.class x J AplikasiP
1_Aplikasi-Perpustakaan > J Member.java > ...
4 public class Member extends Pengguna{
5     private List<Buku> daftarBuku;
6
7     public Member(String idMember, String nama){
8         super(idMember, nama);
9         this.daftarBuku = new ArrayList<>();
10    }
```

Perintah super(idMember, nama) digunakan untuk memanggil constructor dari superclass Pengguna.

Nilai ID dan nama yang diberikan ketika objek Member dibuat akan diteruskan ke constructor Pengguna, kemudian digunakan untuk mengisi atribut id dan nama.

4.4 Penggunaan @Override

Pada class Member terdapat method getName() yang menggunakan annotation @Override.



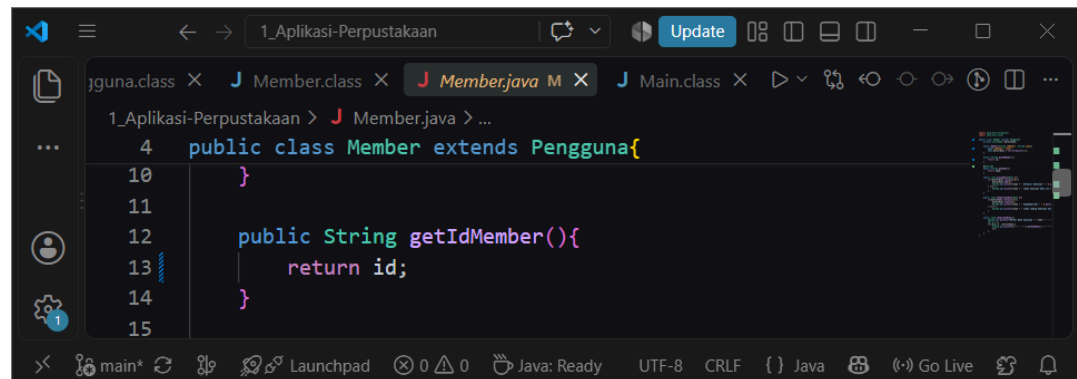
```
guna.class x J Member.class x J Member.java M x J Main.class x
1_Aplikasi-Perpustakaan > J Member.java > ...
4 public class Member extends Pengguna{
15
16     @Override
17     public String getName(){
18         return nama;
19     }
20 }
```

Annotation `@Override` menunjukkan bahwa method tersebut merupakan implementasi ulang dari method `getNama()` yang terdapat pada superclass `Pengguna`.

Penggunaan `@Override` merupakan contoh penerapan konsep overriding, yaitu ketika subclass menyediakan kembali implementasi dari method yang diwarisi dari superclass.

4.5 Method `getIdMember()`

Method `getIdMember()` digunakan untuk mengambil ID yang dimiliki oleh member.

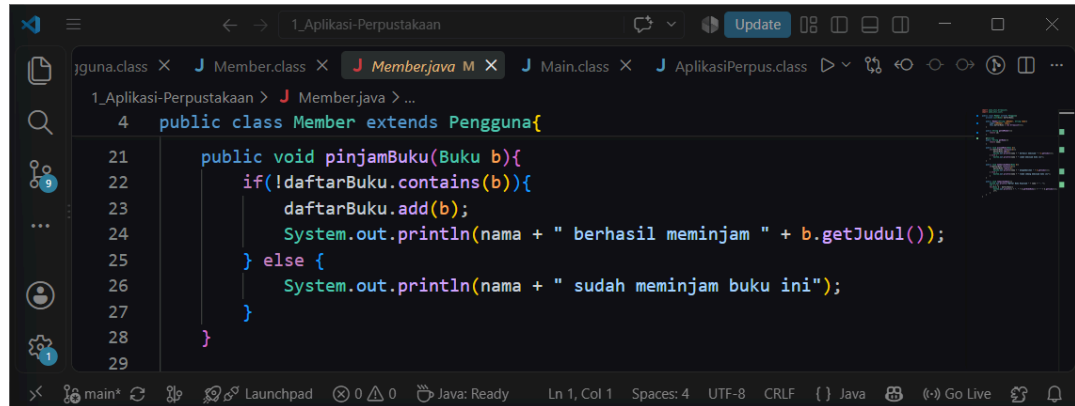


Nilai yang dikembalikan berasal dari atribut `id` yang diwarisi dari class `Pengguna`.

Dengan demikian, class `Member` dapat menggunakan data yang berasal dari superclass melalui inheritance.

4.6 Method `pinjamBuku()`

Method `pinjamBuku()` digunakan untuk mencatat buku yang dipinjam oleh member.



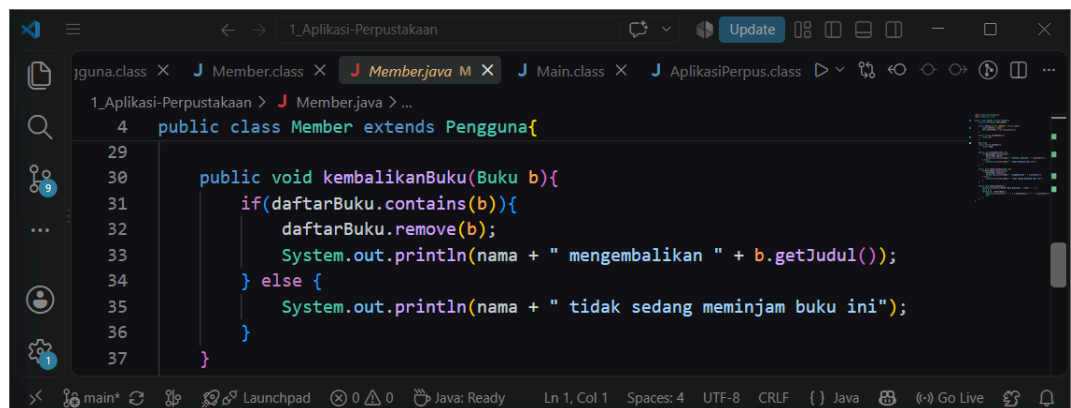
```
1_Aplikasi-Perpustakaan
Update
jguna.class x J Member.class x J Memberjava M x J Main.class x J AplikasiPerpus.class
1_Aplikasi-Perpustakaan > J Member.java > ...
4 public class Member extends Pengguna{
21     public void pinjamBuku(Buku b){
22         if(!daftarBuku.contains(b)){
23             daftarBuku.add(b);
24             System.out.println(nama + " berhasil meminjam " + b.getJudul());
25         } else {
26             System.out.println(nama + " sudah meminjam buku ini");
27         }
28     }
29 }
```

Sebelum menambahkan buku, program melakukan pengecekan untuk memastikan buku tersebut belum ada dalam daftar peminjaman. Jika belum ada, buku akan ditambahkan ke daftarBuku.

Apabila buku sudah tercatat dalam daftar, program akan menampilkan informasi bahwa buku tersebut sebelumnya sudah dipinjam.

4.7 Method kembalikanBuku()

Method kembalikanBuku() digunakan untuk menghapus buku dari daftar buku yang sedang dipinjam.



```
1_Aplikasi-Perpustakaan
Update
jguna.class x J Member.class x J Memberjava M x J Main.class x J AplikasiPerpus.class
1_Aplikasi-Perpustakaan > J Member.java > ...
4 public class Member extends Pengguna{
29
30     public void kembalikanBuku(Buku b){
31         if(daftarBuku.contains(b)){
32             daftarBuku.remove(b);
33             System.out.println(nama + " mengembalikan " + b.getJudul());
34         } else {
35             System.out.println(nama + " tidak sedang meminjam buku ini");
36         }
37     }
}
```

Program terlebih dahulu memeriksa apakah buku tersebut terdapat dalam daftar peminjaman. Jika buku ditemukan, data buku akan dihapus dari daftar.

5. Perubahan Sebelum dan Sesudah Inheritance

Perubahan struktur program setelah penerapan inheritance dapat dilihat pada tabel berikut.

Sebelum Penambahan Inheritance	Setelah Penambahan Inheritance
Belum terdapat class Pengguna	Ditambahkan class Pengguna
Data ID dan nama berada pada Member	Data umum ID dan nama berada pada Pengguna
Member tidak memiliki superclass	Member menjadi subclass dari Pengguna
Tidak terdapat hubungan inheritance	Terdapat hubungan Member extends Pengguna
Constructor Member berdiri sendiri	Constructor Member menggunakan super()
Belum terdapat overriding	Method getNama() menggunakan @Override

6. Link GitHub

https://github.com/AdamasRu/1_Aplikasi-Perpustakaan

7. Kesimpulan

Berdasarkan perubahan yang dilakukan, konsep inheritance telah diterapkan pada aplikasi perpustakaan dengan menggunakan class Pengguna sebagai superclass dan Member sebagai subclass.

Penerapan inheritance dilakukan menggunakan keyword extends, sedangkan keyword super() digunakan untuk memanggil constructor dari superclass. Selain itu, terdapat penggunaan @Override pada method getNama() sebagai penerapan konsep overriding.

Dengan adanya inheritance, struktur program menjadi lebih terorganisir karena atribut dan method yang bersifat umum dapat ditempatkan pada superclass dan digunakan kembali oleh subclass. Sementara itu, fungsi khusus yang berkaitan dengan anggota perpustakaan tetap dikelola oleh class Member.